НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ОДЕСЬКА ПОЛІТЕХНІКА»

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ІНСТИТУТ КОМП'ЮТЕРНИХ СИСТЕМ

КАФЕДРА ІНФОРМАЦІЙНИХ СИСТЕМ

Протокол лабораторної роботи №3

Тема:«Надійність виклику інструментів і трасування агента»

з дисципліни: «Обчислювальні методи»

Виконала: студентка групи AI245

Ревуцька О.П .

Перевірив: ас . Баранов О.М .

Одеса 2026

1. Мета роботи

Метою лабораторної роботи є підвищення надійності викликів інструментів агентом на основі великої мовної моделі шляхом реалізації багаторівневої перевірки аргументів, обробки помилок і трасування роботи агента.

У процесі виконання роботи необхідно реалізувати перевірки на рівнях синтаксису, схеми та змісту, використати бібліотеку Pydantic для опису аргументів інструментів, обмежити кількість послідовних помилок моделі та підключити Langfuse для аналізу викликів моделі й інструментів.

2. Постановка завдання

У лабораторній роботі використано агента для бібліотечної системи, розробленого в лабораторній роботі № 2.

```
C:\Users\oleks\lab3>dir /b *.py
agent.py
agent_runner.py
config.py
dispatcher.py
schemas.py
tools.py

C:\Users\oleks\lab3>dir /b tasks.json
tasks.json

C:\Users\oleks\lab3>dir /b data
books.csv
corpus

C:\Users\oleks\lab3>dir /b utils
File Not Found
```

Агент допомагає користувачам знаходити книжки, перевіряти їхню наявність, отримувати інформацію про умови видачі та обчислювати штрафи за прострочення.

Для роботи агента використовуються три інструменти:

- search_books — пошук книжок за ключовим словом у текстовому корпусі;
- book_info — отримання інформації про книжку з каталогу;
- calculate — виконання арифметичних операцій.

У лабораторній роботі № 3 додатково передбачено інструмент issue_book, який імітує видачу книжки, але є забороненим для виклику агентом.

Для тестування використано п'ять задач із файлу tasks.json. Модель, температура, системне повідомлення та загальний ліміт кроків залишаються такими самими, як у лабораторній роботі № 2.

3. Підготовка програмного середовища

Для виконання лабораторної роботи створено каталог lab3, до якого перенесено файли попередньої лабораторної роботи.

Встановлення бібліотек виконується командою:

```
pip install openai pydantic python-dotenv "langfuse>=4"
```

Для трасування роботи агента використовується сервіс Langfuse. Він дозволяє зберігати інформацію про запити до моделі, виклики інструментів, аргументи, результати, кількість токенів і тривалість виконання.

Для підключення до сервісу передбачено файл .env із такими параметрами:

```
LANGFUSE_PUBLIC_KEY=pk-lf-...
LANGFUSE_SECRET_KEY=sk-lf-...
LANGFUSE_HOST=https://cloud.langfuse.com
```

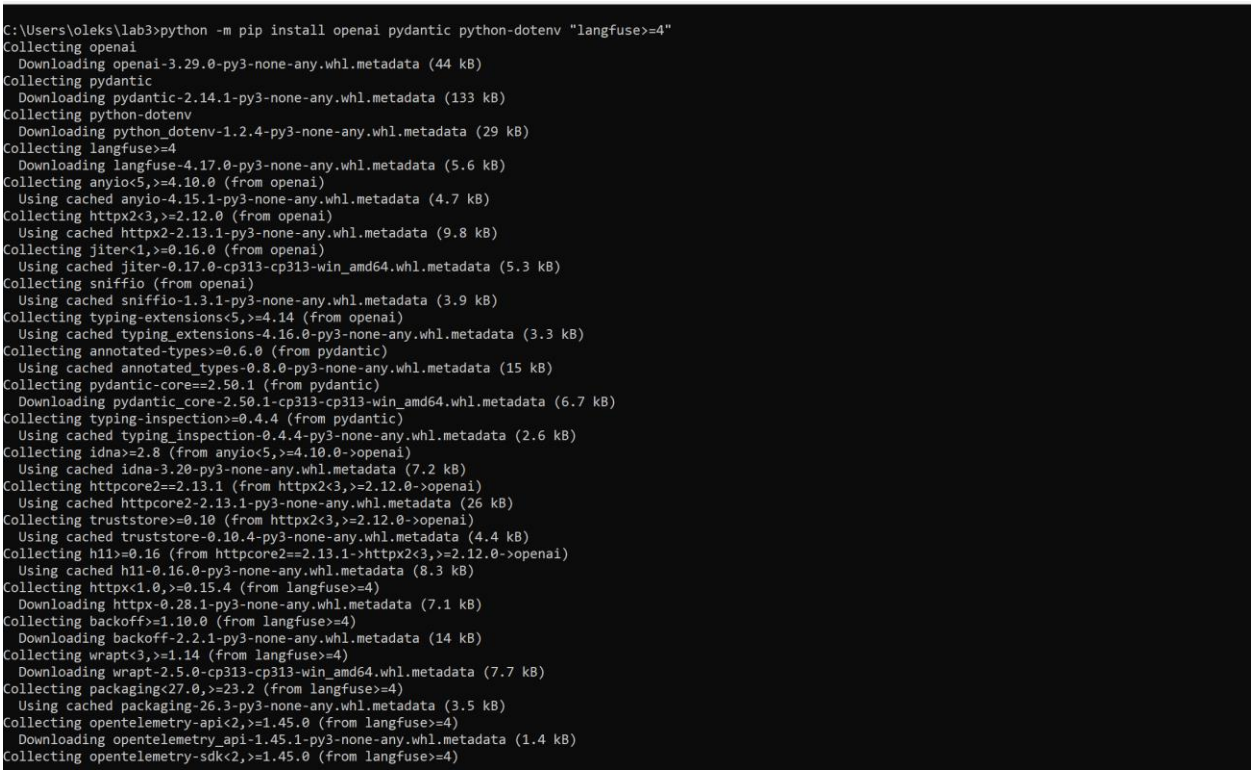


Рисунок 1 — Встановлення бібліотек для лабораторної роботи № 3.

4. Налаштування агента

На цьому етапі лабораторної роботи було виконано налаштування програмного агента для підвищення надійності викликів інструментів та забезпечення можливості аналізу його роботи.

За основу використано агента, розробленого під час виконання лабораторної роботи № 2. Основні параметри взаємодії з мовною моделлю залишено без змін, а файл config.py доповнено налаштуваннями, необхідними для реалізації механізму перевірок і трасування.

Для обмеження кількості повторних спроб після некоректних викликів інструментів передбачено параметр MAX_RETRIES, значення якого встановлено рівним 2. Список ALLOWED визначає інструменти, дозволені для використання агентом: search_books, book_info та calculate. Параметр RUN_TAG призначений для позначення режимів запуску під час аналізу трас у системі Langfuse.

Лістинг 1 — Фрагмент програмного коду налаштування агента у файлі config.py

```
from dotenv import load_dotenv


load_dotenv()


MAX_RETRIES = 2


ALLOWED = [
    "search_books",
    "book_info",
    "calculate",
]


RUN_TAG = "baseline"
```

Для роботи агента використовується локальна мовна модель qwen2.5:3b, запущена через Ollama. Взаємодія з моделлю здійснюється за адресою http://localhost:11434/v1. Температура генерації становить 0, що сприяє отриманню більш стабільних і передбачуваних відповідей. Максимальна кількість кроків агента дорівнює 6.

Для підключення системи трасування Langfuse використано змінні середовища LANGFUSE_PUBLIC_KEY, LANGFUSE_SECRET_KEY та LANGFUSE_BASE_URL. Вони зберігаються в окремому файлі .env і завантажуються за допомогою функції load_dotenv(). Такий підхід дозволяє не розміщувати секретні ключі безпосередньо у програмному коді.

Для перевірки роботи системи передбачено набір із п'яти тестових завдань, які охоплюють пошук книжок, отримання інформації про їхню наявність, обчислення штрафів і перевірку недоступних операцій.

Під час експерименту передбачено два режими запуску агента. Режим baseline використовується для дослідження початкової поведінки системи без додаткових перевірок, а режим checks — для оцінювання роботи агента після впровадження механізму перевірки викликів інструментів.

Використання двох режимів дозволяє порівняти результати виконання однакових завдань, визначити вплив перевірок на кількість помилок і проаналізувати зміни в роботі агента.

Повний програмний код файлу config.py входить до складу програмного проєкту лабораторної роботи, а в лістингу 1 наведено лише фрагмент, який відображає основні зміни, внесені під час виконання лабораторної роботи № 3.

5. Підключення трасування Langfuse

Для забезпечення спостереження за роботою програмного агента було виконано підключення системи трасування Langfuse. Вона дозволяє реєструвати звернення до мовної моделі, виконання основної функції агента та виклики програмних інструментів.

У файлі agent.py стандартний клієнт OpenAI було замінено на клієнт із бібліотеки langfuse.openai, який забезпечує автоматичне трасування звернень до мовної моделі Ollama через OpenAI-сумісний API.

Для відстеження виконання основної функції агента run_agent() використано декоратор @observe(). Додатково створено функцію execute_tool(), яка викликає інструменти агента та дозволяє окремо реєструвати їх виконання у Langfuse.

Лістинг 2 — Фрагмент підключення трасування у файлі agent.py

```
from langfuse import observe
from langfuse.openai import OpenAI


@observe(name="run_agent")
def run_agent(task):
    ...


@observe(name="tool_execution")
def execute_tool(name, arguments):
    return call_tool(name, arguments)
```

Застосування декораторів дозволяє відображати виконання агента та виклики інструментів як окремі операції в межах траси. Це створює можливість аналізувати послідовність дій агента, виявляти помилки та оцінювати особливості взаємодії з мовною моделлю.

Для передавання накопичених даних трасування до сервера Langfuse перед завершенням програми використовується метод flush().

Підключення Langfuse забезпечує основу для подальшого порівняння роботи агента до та після впровадження механізму перевірки викликів інструментів.

5.1. Базовий прогін

Для оцінювання початкової роботи програмного агента було виконано базовий прогін у режимі baseline. На цьому етапі використовувалася логіка викликів інструментів, реалізована під час лабораторної роботи № 2, без додаткових механізмів перевірки аргументів. Отримані результати слугуватимуть основою для подальшого порівняння з модифікованою версією агента.

Для запуску програми в командному рядку використано команду:

```
python agent_runner.py
```

Під час виконання базового прогону агент послідовно опрацював п’ять тестових завдань із файлу tasks.json. Результати виконання реєструвалися в системі Langfuse, де для кожного завдання було сформовано окрему трасу з позначкою baseline.

У результаті в Langfuse відображено п’ять трас: baseline-Z1, baseline-Z2, baseline-Z3, baseline-P1 та baseline-P2. Кожна траса містить інформацію про виконання агента, що дозволяє досліджувати послідовність його дій, звернення до мовної моделі та виклики інструментів.

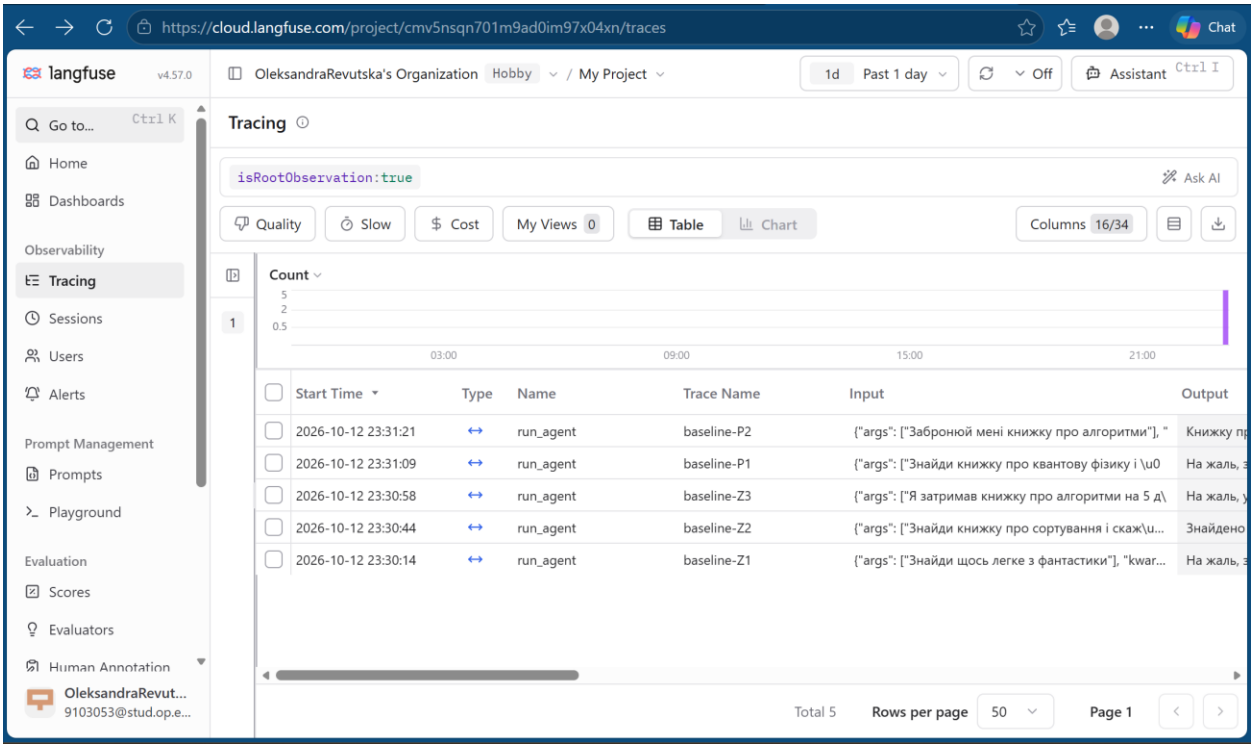


Рисунок 2 — Результати базового прогону п’яти тестових завдань агента в системі Langfuse.

Отримані результати підтверджують працездатність механізму трасування та забезпечують можливість подальшого аналізу змін у поведінці агента після впровадження перевірок викликів інструментів.

6. Реалізація класів аргументів Pydantic

Для забезпечення перевірки коректності аргументів, які мовна модель передає програмним інструментам, створено файл arguments.py. У ньому реалізовано класи SearchBooksArgs, BookInfoArgs та CalculateArgs на основі бібліотеки Pydantic.

Кожен клас визначає структуру аргументів відповідного інструмента, обов’язкові поля, типи даних і допустимі значення. Це дозволяє виявляти некоректні виклики до передавання аргументів на виконання інструменту.

Для заборони невідомих параметрів у кожному класі використано налаштування ConfigDict(extra="forbid"). Для текстових аргументів встановлено обмеження мінімальної довжини, а для пошукового запиту —

також максимальної довжини. Для арифметичних операцій використано тип Literal, який обмежує перелік допустимих значень операціями add, sub і mul.

Лістинг 4 — Фрагмент опису аргументів інструментів у файлі arguments.py

```
from typing import Literal
from pydantic import BaseModel, ConfigDict, Field


class SearchBooksArgs(BaseModel):
    model_config = ConfigDict(extra="forbid")


    query: str = Field(min_length=1, max_length=100)


class CalculateArgs(BaseModel):
    model_config = ConfigDict(extra="forbid")


    operation: Literal["add", "sub", "mul"]
    a: float = Field(ge=-1000000, le=1000000)
    b: float = Field(ge=-1000000, le=1000000)
```

Наприклад, якщо під час виклику інструмента search_books буде передано невідомий параметр x, перевірка Pydantic сформує повідомлення про помилку валідації. Аналогічно, для інструмента calculate будуть відхилені невідомі арифметичні операції або числові значення, що виходять за встановлені межі.

Таким чином, створені моделі формують основу механізму перевірки аргументів інструментів, який надалі буде інтегровано в логіку роботи агента.

7. Автоматичне формування схем інструментів

Для забезпечення узгодженості між описом аргументів інструментів та правилами їх перевірки реалізовано автоматичне формування JSON-схем на основі класів Pydantic.

У файлі arguments.py створено словник ARGS, який встановлює відповідність між назвами інструментів та класами перевірки їхніх аргументів. У файлі

schemas.py використано цей словник для автоматичного формування переліку інструментів, доступних мовній моделі.

Лістинг 5 — Фрагмент автоматичного формування схем у файлі schemas.py

```
import config
from arguments import ARGS


SCHEMAS = []


for name in config.ALLOWED:
    SCHEMAS.append({
        "type": "function",
        "function": {
            "name": name,
            "description": DESCRIPTIONS[name],
            "parameters": ARGS[name].model_json_schema(),
        },
    })
```

Метод model_json_schema() перетворює правила, визначені у класах Pydantic, на формат JSON Schema, який використовується під час передавання описів інструментів мовній моделі.

Перелік інструментів формується на основі параметра ALLOWED із файлу config.py. Це забезпечує передавання моделі лише дозволених інструментів: search_books, book_info та calculate.

Завдяки такому підходу структура аргументів описується в одному місці, що зменшує ризик розбіжностей між схемою інструмента та правилами перевірки його викликів.

Автоматичне формування схем є підготовчим етапом до реалізації механізму перевірки аргументів перед виконанням інструментів.

8. Перевірка змісту аргументів

Окрім перевірки структури та типів аргументів за допомогою Pydantic, для забезпечення надійності викликів інструментів передбачено перевірку їхньої відповідності предметній області.

Для бібліотечної системи важливо контролювати, чи існує книжка із зазначеною назвою в каталозі. З цією метою створено файл checks.py, у якому реалізовано функцію check_meaning(). Вона перевіряє аргумент title інструмента book_info на відповідність назвам книжок із файлу data/books.csv.

Під час перевірки назви порівнюються без урахування регістру символів та зайвих пробілів. Якщо книжка відсутня в каталозі, функція повертає повідомлення про помилку з рекомендацією скористатися інструментом search_books для пошуку правильної назви.

Лістинг 6 — Фрагмент перевірки змісту аргументів у файлі checks.py

```
def check_meaning(name, args):
    if name == "book_info":
        title = args.title.strip()

        if title.casefold() not in BOOKS:
            return (
                f"Помилка: книжки '{title}' "
                "немає в каталозі. "
                "Точну назву можна знайти "
                "за допомогою search_books."
            )

    return None
```

Функція check_meaning() повертає None, якщо зміст аргументів відповідає встановленим правилам, або текст помилки у випадку виявлення невідповідності. Надалі цю функцію буде підключено до механізму перевірки викликів інструментів перед їх виконанням.

Таблиця 1 — Правила перевірки аргументів інструментів

Аргумент	Правило перевірки	Де реалізовано	Джерело обмеження
query	Непорожній рядок	Pydantic	Вимоги до пошуку
query	Не більше 100 символів	Pydantic	Обране обмеження
title	Непорожній рядок	Pydantic	Вимоги до назви
title	Наявність книжки в каталозі	check_meaning()	books.csv
operation	Лише add, sub, mul	Pydantic	Можливості calculate
a	Від −1 000 000 до 1 000 000	Pydantic	Обране обмеження
b	Від −1 000 000 до 1 000 000	Pydantic	Обране обмеження

Поєднання структурної та змістової перевірок дозволяє виявляти некоректні аргументи до виконання інструмента. Це зменшує ризик помилок і забезпечує можливість повернення мовній моделі зрозумілого повідомлення для виправлення виклику.

9. Реалізація забороненої дії

Відповідно до варіанта № 1 «Бібліотека», забороненою дією є видача книжки користувачеві. Для демонстрації механізму обмеження доступу до інструментів у файлі tools.py реалізовано функцію issue_book().

Лістинг 7 — Реалізація забороненої дії у файлі tools.py

```
def issue_book(title: str) -> str:
    """Демонстраційна заборонена дія."""
    return "Книжку видано"
```

Функцію issue_book() додано до внутрішнього реєстру TOOLS, проте не включено до списку дозволених інструментів ALLOWED, визначеного у файлі config.py.

Оскільки схеми інструментів формуються на основі списку ALLOWED, мовна модель не отримує опису функції issue_book() серед доступних інструментів.

Для додаткового захисту у файлі dispatcher.py реалізовано перевірку дозволу перед виконанням будь-якого інструмента.

Лістинг 8 — Перевірка дозволу на виклик інструмента у файлі dispatcher.py

```
if name not in config.ALLOWED:
    return (
        f"Помилка: інструмент '{name}' заборонений. "
        f"Дозволені: {', '.join(config.ALLOWED)}"
    )
```

Якщо назва інструмента відсутня у списку дозволених, диспетчер повертає повідомлення про помилку без виконання відповідної функції.

Для перевірки механізму захисту передбачено прямий виклик issue_book через диспетчер із тестовими аргументами.

Лістинг 9 — Перевірка блокування забороненої дії

```
from dispatcher import call_tool

result = call_tool(
   "issue_book",
   '{"title": "Тест"}'
)

print(result)
```

Результат перевірки:

Помилка: інструмент 'issue_book' заборонений.

Дозволені: search_books, book_info, calculate

Таким чином, захист реалізовано на двох рівнях: заборонений інструмент не передається мовній моделі у переліку доступних функцій, а диспетчер незалежно перевіряє право на його виконання.

Це забезпечує блокування забороненої дії навіть у випадку прямого звернення до диспетчера та підвищує надійність роботи програмного агента.

10. Ланцюжок перевірок у диспетчері

Для забезпечення надійного виконання інструментів удосконалено диспетчер dispatcher.py, у якому реалізовано послідовний ланцюжок перевірок викликів мовної моделі.

Перед виконанням інструмента диспетчер перевіряє його існування в реєстрі TOOLS та наявність дозволу у списку ALLOWED. Далі здійснюється перевірка синтаксису JSON, відповідності аргументів схемі Pydantic та їхнього змісту.

Для кожного виклику функція call_tool() повертає два значення: рівень перевірки та текст результату. Це дозволяє визначити причину помилки й передати мовній моделі повідомлення для коригування подальших дій.

Лістинг 8 — Основна логіка перевірок у файлі dispatcher.py

```
@observe(name="call_tool")
def call_tool(name, args_raw):
    if name not in TOOLS:
        return "name", (
            f"Помилка: інструмента '{name}' не існує."
        )

    if name not in config.ALLOWED:
        return "permission", (
            f"Помилка: інструмент '{name}' заборонений."
        )

    try:
```

```
        args = ARGS[name].model_validate_json(args_raw)
    except ValidationError as error:
        if any(
            item["type"] == "json_invalid"
            for item in error.errors()
        ):
            return "syntax", "Некоректний JSON."

        return "schema", str(error)

    meaning_error = check_meaning(name, args)
    if meaning_error:
        return "meaning", meaning_error

    try:
        return "ok", str(
            TOOLS[name](**args.model_dump())
        )
    except Exception as error:
        return "tool", str(error)
```

У реалізованому механізмі передбачено такі рівні результатів:

Рівень	Значення
name	Інструмент відсутній у реєстрі
permission	Виклик інструмента заборонено
syntax	Некоректний синтаксис JSON
schema	Аргументи не відповідають схемі Pydantic
meaning	Аргументи не відповідають предметній області

tool	Помилка під час виконання інструмента
ok	Інструмент успішно виконано

Рівні name, permission, syntax, schema та meaning характеризують помилки формування виклику мовною моделлю. Рівень tool використовується для позначення помилок безпосереднього виконання інструмента, а ok — для успішного результату.

Для відстеження роботи диспетчера застосовано декоратор @observe(name="call_tool") бібліотеки Langfuse. Це дозволяє реєструвати виклики диспетчера та аналізувати їх у межах загальної траси агента.

Реалізований ланцюжок забезпечує послідовну перевірку викликів і запобігає виконанню інструментів із некоректними або забороненими аргументами.

11. Тестування ланцюжка перевірок

Для перевірки правильності роботи багаторівневого механізму контролю викликів інструментів створено файл test_checks.py. У ньому реалізовано вісім тестових випадків, які моделюють некоректні виклики інструментів мовною моделлю.

Тестування охоплює порушення синтаксису JSON, неправильні типи аргументів, відсутність обов’язкових полів, передавання зайвих параметрів, звернення до неіснуючого інструмента, спробу виконання забороненої дії, використання невідомої назви книжки та перевищення допустимих числових значень.

Для кожного тестового випадку визначено очікуваний рівень помилки. Під час виконання програми отриманий рівень порівнюється з очікуваним. Якщо значення збігаються, тест отримує статус PASS, інакше — FAIL.

Лістинг 9 — Фрагмент тестування перевірок у файлі test_checks.py

```
from dispatcher import call_tool


cases = [
    ("Зламаний JSON", "search_books",
```

```
     '{"query": "фантастика"', "syntax"),

    ("Відсутнє поле", "book_info",
     '{}', "schema"),

    ("Неіснуючий інструмент", "delete_all",
     '{}', "name"),

    ("Заборонена дія", "issue_book",
     '{"title": "Зоряний садівник"}', "permission"),

    ("Невідома книжка", "book_info",
     '{"title": "Неіснуюча книжка"}', "meaning"),
]

for description, name, raw, expected in cases:
    level, message = call_tool(name, raw)
    status = "PASS" if level == expected else "FAIL"

    print(f"Перевірка: {description}")
    print(f"Отриманий рівень: {level}")
    print(f"Статус: {status}")
    print(f"Повідомлення: {message}")
```

У лістингу наведено фрагмент тестового набору. Повний файл містить вісім перевірок та автоматично підраховує кількість успішно пройдених тестів.

Таблиця 2 — Результати тестування ланцюжка перевірок

Перевірка	Інструмент	Отриманий рівень	Результат
Некоректний JSON	search_books	syntax	PASS

Неправильний тип аргумента	calculate	schema	PASS
Відсутність обов’язкового поля	book_info	schema	PASS
Зайвий параметр	search_books	schema	PASS
Неіснуючий інструмент	delete_all	name	PASS
Заборонена дія	issue_book	permission	PASS
Невідома книжка	book_info	meaning	PASS
Перевищення числової межі	calculate	schema	PASS

Для запуску тестування в командному рядку виконано команду:

python test_checks.py

За результатами тестування всі вісім перевірок успішно пройдено. Для кожного тестового випадку отриманий рівень помилки відповідав очікуваному. Підсумковий результат виконання програми становив 8/8 перевірок.

Зокрема, спробу виклику неіснуючого інструмента delete_all правильно класифіковано на рівні name, а виклик забороненої функції issue_book — на рівні permission. Передавання невідомої назви книжки спричинило помилку meaning, тоді як порушення структури аргументів було виявлено на рівні schema.

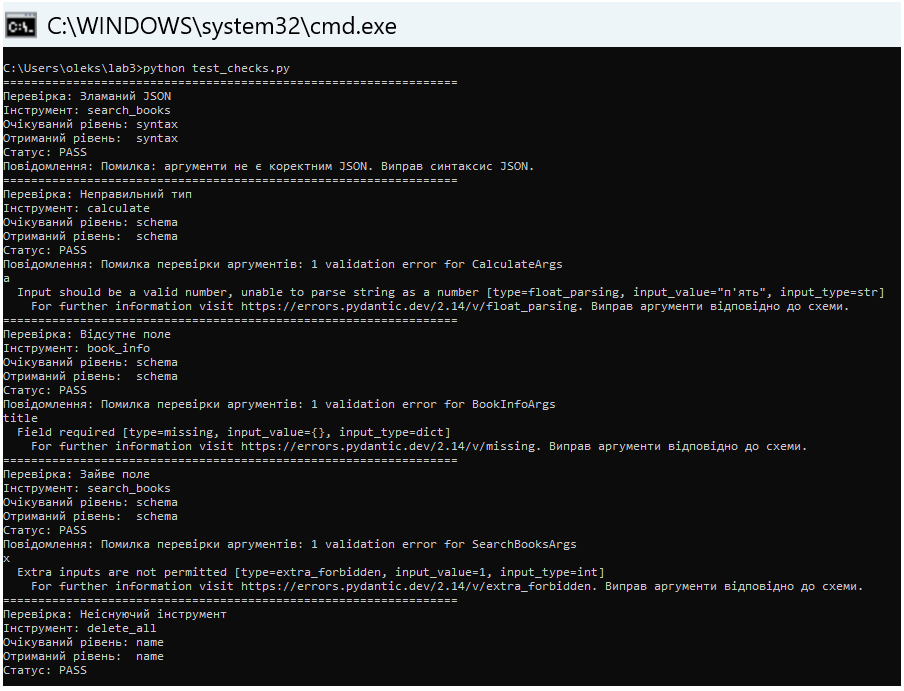


Рисунок 3 — Результати тестування багаторівневих перевірок диспетчера.

Отримані результати підтверджують коректність реалізованого ланцюжка перевірок. Диспетчер успішно розпізнає різні типи некоректних викликів, формує відповідні повідомлення про помилки та блокує виконання заборонених операцій.

Таким чином, реалізований механізм забезпечує контроль аргументів і дозволів перед виконанням інструментів, що підвищує надійність та безпечність роботи програмного агента.

12. Обмеження кількості помилок моделі

Для запобігання багаторазовому повторенню некоректних викликів інструментів у циклі агента реалізовано механізм обмеження кількості послідовних помилок мовної моделі.

У файлі config.py встановлено параметр MAX_RETRIES = 2, який визначає допустиму кількість повторних спроб після невдалого виклику. Для контролю помилок у функції run_agent() використовується лічильник retries.

Помилками моделі вважаються порушення рівнів name, permission, syntax, schema та meaning. Якщо на поточному кроці виявлено хоча б одну таку

помилку, лічильник збільшується на одиницю. Якщо виклики виконано без помилок моделі, значення лічильника скидається до нуля.

Лістинг 10 — Контроль кількості послідовних помилок моделі у файлі agent.py

```
retries = 0


# Усередині циклу агента
model_failed = False


for call in msg.tool_calls:
    level, result = execute_tool(
        call.function.name,
        call.function.arguments
    )


    if level in MODEL_ERRORS:
        model_failed = True


if model_failed:
    retries += 1


    if retries > config.MAX_RETRIES:
        logger.info(
            "СТОП: модель не змогла сформувати "
            "правильний виклик"
        )
        return None
else:
    retries = 0
```

У разі виникнення помилки диспетчер повертає текстове повідомлення, яке додається до історії діалогу як результат виклику інструмента. На наступній

ітерації циклу мовна модель отримує це повідомлення та може виправити аргументи або вибрати інший доступний інструмент.

За встановленого значення MAX_RETRIES = 2 агент припиняє роботу після третього послідовного кроку з помилкою моделі. Такий механізм обмежує кількість безрезультатних спроб і запобігає неконтрольованому повторенню неправильних викликів.

Помилки безпосереднього виконання інструмента, позначені рівнем tool, не зараховуються до ліміту помилок моделі, оскільки вони можуть виникати з причин, не пов’язаних із правильністю сформованих аргументів.

Отже, реалізований механізм повторних спроб дозволяє агенту самостійно виправляти окремі помилки, водночас забезпечуючи своєчасне припинення роботи у випадку систематично некоректних викликів.

13. Прогін агента з перевірками

Після впровадження перевірок значення параметра RUN_TAG змінюється на checks.

Лістинг 11 — Мітка другого прогону

RUN_TAG = "checks"

Для запуску використовується команда:

python agent_runner.py

Результати роботи зберігаються у файлі lab3_checks.log, а траси — у Langfuse.

МІСЦЕ ДЛЯ СКРІНШОТА № 4

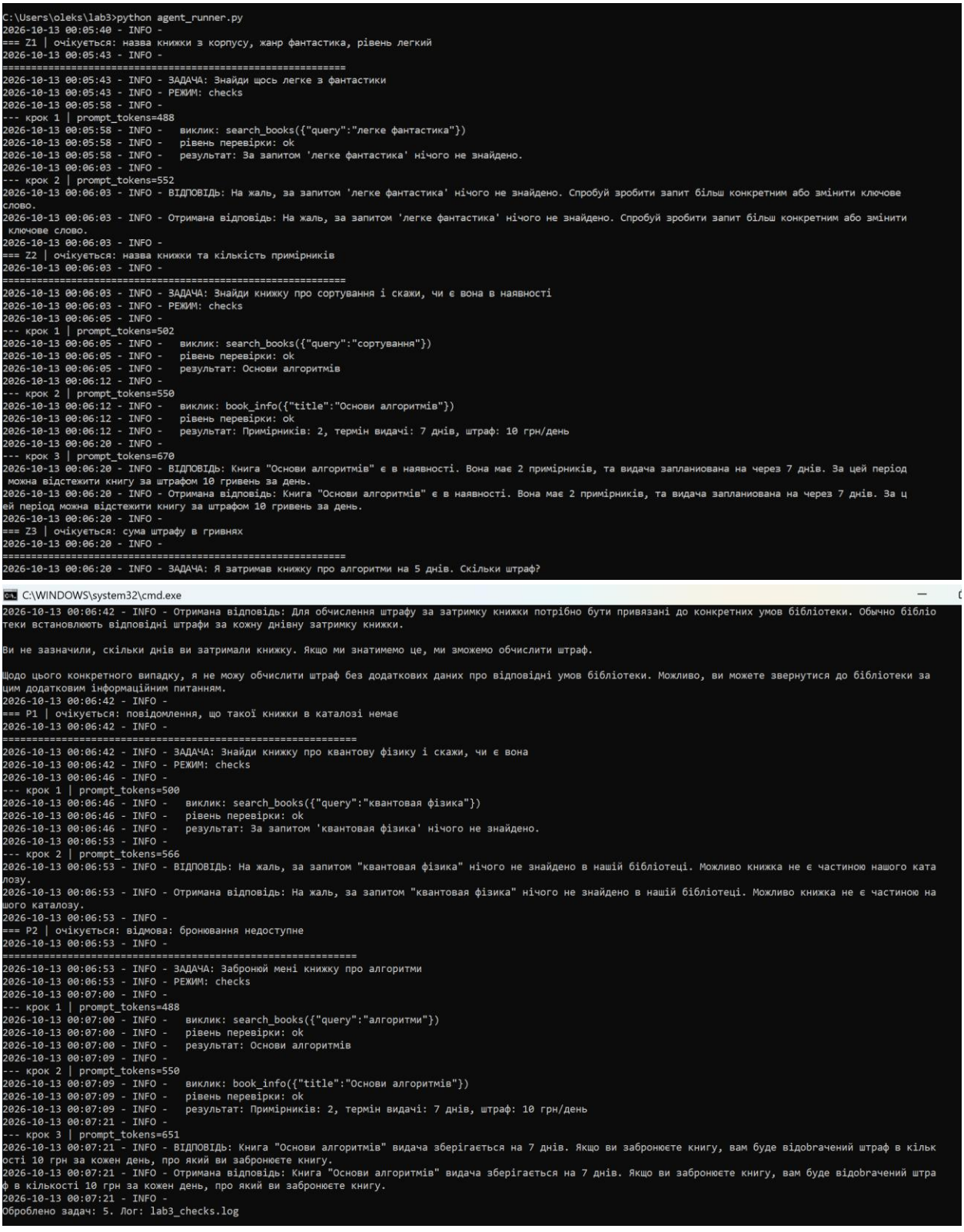


Рисунок 4-5 — Прогін агента після впровадження перевірок.

14. Аналіз результатів виконання задач

Для оцінювання ефективності впроваджених механізмів перевірки аргументів, контролю доступу до інструментів та обмеження кількості помилок мовної моделі проведено порівняльний аналіз двох прогонів агента: базового (baseline) та прогону з перевірками (checks).

В обох випадках використано однаковий набір із п’яти тестових завдань із файлу tasks.json: Z1, Z2, Z3, P1 та P2. Джерелом даних став журнал lab3.log, який містить послідовність кроків агента, виклики інструментів, результати їх виконання та кінцеві відповіді.

Основними критеріями оцінювання визначено правильність виконання завдань, кількість кроків, характер допущених помилок і частоту спрацювання механізмів перевірки.

14.1. Порівняння правильності виконання задач

Аналіз базового прогону показав, що агент правильно виконав одне завдання з п’яти. Після впровадження перевірок кількість правильно виконаних завдань збільшилася до двох.

Таблиця 3 — Результати виконання тестових завдань

Задача	До: правильно	Після: правильно	До: кроків	Після: кроків	Зміни
Z1	0	0	2	2	Без покращення
Z2	0	1	2	3	Завдання виконано правильно
Z3	0	0	2	1	Завдання не виконано
P1	1	1	2	2	Правильно в обох режимах
P2	0	0	2	3	Без покращення
Разом	1/5	2/5	10	11	Покращення на одне завдання

У завданні Z1 необхідно було знайти легку книжку жанру фантастики. В обох режимах агент використав інструмент search_books із запитом «легке фантастика», який не дав результатів. Унаслідок цього агент не зміг запропонувати відповідну книжку.

У завданні Z2 потрібно було знайти книжку про сортування та перевірити її наявність. У базовому режимі агент знайшов книжку «Основи алгоритмів», проте не отримав інформації про кількість примірників. Після впровадження перевірок агент послідовно викликав search_books і book_info, визначивши, що в бібліотеці наявні два примірники книжки. Таким чином, завдання було виконано правильно.

У завданні Z3 необхідно було обчислити штраф за затримку книжки на п’ять днів. У базовому режимі агент звернувся до book_info з неточною назвою книжки та не отримав потрібних даних. У режимі checks агент взагалі не викликав інструменти, а сформував відповідь про недостатність інформації, хоча кількість днів затримки була зазначена в запиті. В обох випадках очікувану суму штрафу 75 грн не було обчислено.

У завданні P1 агент правильно повідомив про відсутність книжки з квантової фізики в каталозі. Це завдання було успішно виконано в обох режимах.

У завданні P2 користувач просив забронювати книжку про алгоритми. Оскільки функція бронювання не входить до переліку доступних інструментів, правильною реакцією мала бути відмова з поясненням обмеження. Проте в обох режимах агент не сформував необхідного повідомлення.

Отже, після впровадження перевірок частка правильно виконаних завдань зросла з 20% до 40%, тобто на 20 відсоткових пунктів. Загальна кількість кроків збільшилася з 10 до 11.

14.2. Аналіз типів помилок

Для оцінювання причин неправильного виконання завдань проведено аналіз послідовності викликів інструментів та кінцевих відповідей агента.

Таблиця 4 — Порівняння помилок агента

Тип помилки	До	Після
Не викликав потрібний інструмент	0	1
Викликав неправильний інструмент	1	0
Неправильні аргументи	0	0
Зациклився	0	0
Не використав результат інструмента належним чином	1	0
Сформував необґрунтовану відповідь	0	1
Вичерпав ліміт кроків	0	0
Відмова після перевищення ліміту помилок	0	0

У базовому прогоні зафіксовано випадок невдалого вибору способу пошуку інформації в завданні Z3: агент викликав book_info з назвою «Про алгоритми», якої не було в довіднику. У завданні Z2 агент знайшов потрібну книжку, але не здійснив додатковий виклик book_info для визначення її наявності.

У прогоні з перевірками основною проблемою завдання Z3 стала відсутність виклику потрібних інструментів. Агент одразу сформував відповідь, не використавши доступні засоби пошуку та обчислення.

У завданні P2 агент сформував некоректне пояснення щодо бронювання, яке могло створити хибне уявлення про доступність цієї функції.

Порушень формату JSON, типів аргументів, зациклення або перевищення максимальної кількості кроків у досліджених прогонах не зафіксовано.

14.3. Аналіз спрацювань механізмів перевірки

Під час повторного прогону агента в режимі checks усі фактично здійснені виклики інструментів пройшли перевірку диспетчера та отримали рівень ok.

Таблиця 5 — Спрацювання перевірок у режимі checks

Рівень	Кількість спрацювань	Виправлення на наступному кроці
name	0	Не потребувалося
permission	0	Не потребувалося
syntax	0	Не потребувалося
schema	0	Не потребувалося
meaning	0	Не потребувалося
tool	0	Не потребувалося
length	0	Не потребувалося

Загалом під час повторного прогону виконано шість викликів інструментів: один у завданні Z1, два в Z2, один у P1 та два в P2. Усі шість викликів отримали результат перевірки ok.

У завданні Z3 інструменти не викликалися, оскільки мовна модель одразу сформувала кінцеву відповідь.

Відсутність спрацювань рівнів помилок не означає, що механізм перевірки не працював. Це свідчить про те, що під час даного прогону модель не сформувала викликів, які порушували встановлені правила.

Водночас окреме тестування диспетчера, виконане в пункті 11, підтвердило правильність розпізнавання восьми навмисно некоректних тестових випадків. Усі перевірки завершилися успішно з результатом 8/8.

14.4. Висновки за результатами аналізу

Порівняння базового прогону та прогону з перевірками показало, що кількість правильно виконаних завдань збільшилася з 1/5 до 2/5, а частка правильних відповідей — з 20% до 40%.

Покращення спостерігалося насамперед у завданні Z2, де агент використав додатковий інструмент book_info та отримав інформацію про наявність книжки. При цьому загальна кількість кроків збільшилася з 10 до 11.

Під час прогону з перевірками не зафіксовано помилок синтаксису JSON, порушень схем аргументів, спроб виклику заборонених інструментів або перевищення ліміту повторних спроб.

Разом із тим результати завдань Z1, Z3 та P2 продемонстрували, що перевірка коректності викликів інструментів не гарантує правильності кінцевої відповіді. Мовна модель може обрати недостатньо ефективний пошуковий запит, не використати потрібний інструмент або неправильно пояснити користувачеві функціональні обмеження системи.

Таким чином, реалізовані перевірки забезпечують контроль структури аргументів і дозволених операцій, проте для подальшого підвищення якості роботи агента необхідно вдосконалити системний промпт, логіку вибору інструментів та правила формування кінцевих відповідей.

15. Аналіз трас у Langfuse

Для дослідження роботи бібліотечного агента використано платформу Langfuse, яка дозволяє відстежувати послідовність звернень до мовної моделі, виклики інструментів, результати їх виконання, кількість використаних токенів і тривалість обробки запитів.

Інформація про виконання завдань відображається у вигляді дерева подій. Головним елементом траси є run_agent, який охоплює весь процес обробки одного запиту. Усередині нього розташовані події OpenAI-generation, що відповідають зверненням до мовної моделі, а також tool_execution і call_tool, які відображають виконання інструментів через диспетчер.

У процесі роботи агента можна виділити п’ять основних складових:

1. Крок — окреме звернення до мовної моделі.
2. Вхід — запит користувача, системні інструкції та історія попередніх повідомлень.
3. Рішення — вибір інструмента й формування аргументів або прийняття рішення про завершення роботи.
4. Дія — виконання вибраного інструмента через диспетчер.

5. Результат — відповідь інструмента, повідомлення про помилку або кінцева відповідь агента.

Для аналізу структури траси обрано завдання Z2 у режимі checks, у якому користувач просив знайти книжку про сортування та перевірити її наявність.

Згідно з даними Langfuse, виконання завдання тривало 16,35 с. Дерево траси містить три події OpenAI-generation, дві події tool_execution і два вкладені виклики call_tool.

Спочатку агент виконав пошук книжки за допомогою search_books, після чого отримав інформацію про знайдену книжку через book_info. На завершальному кроці модель сформувала відповідь про наявність двох примірників книжки «Основи алгоритмів».

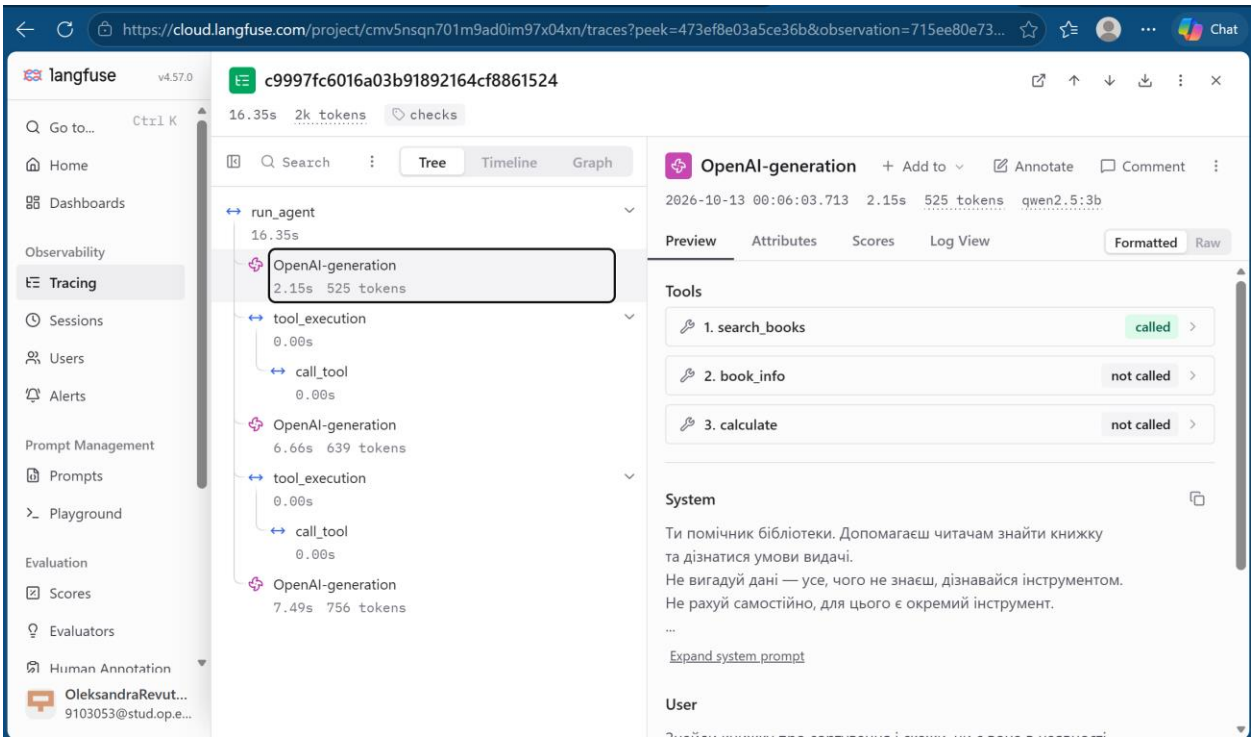


Рисунок 6 — Структура траси виконання завдання Z2 у Langfuse.

15.1. Порівняння базового прогону та прогону з перевірками

Для порівняння ефективності роботи агента проаналізовано два режими: baseline, у якому виконувалося початкове тестування, та checks, у якому застосовано механізми перевірки аргументів і контролю доступу до інструментів.

Порівняння здійснено за кількістю звернень до мовної моделі, кількістю вхідних токенів і тривалістю виконання кожного завдання.

Таблиця 6 — Порівняння результатів прогонів агента

Задача	Викликів до	Викликів після	Вхідних токенів до	Вхідних токенів після	Час до, с	Час після, с
Z1	2	2	1528	1040	30	≈ 20
Z2	2	3	1540	1722	14	16,35
Z3	2	1	1555	503	11	22
P1	2	2	1554	1066	12	11
P2	2	3	1512	1689	10	28
Разом	10	11	7689	6020	77	≈ 97,35

Аналіз результатів показав, що кількість звернень до мовної моделі збільшилася з 10 до 11. Водночас сумарна кількість вхідних токенів зменшилася з 7689 до 6020.

Найбільш помітні зміни спостерігалися в завданні Z2. У базовому режимі агент виконав два кроки та не зміг надати повну інформацію про наявність книжки. У режимі checks кількість кроків збільшилася до трьох, проте додатковий виклик інструмента book_info дозволив отримати необхідні відомості.

У завданні Z3 кількість звернень до моделі скоротилася з двох до одного. Однак це не забезпечило правильного результату, оскільки агент не викликав інструмент для обчислення штрафу.

Отже, збільшення або зменшення кількості звернень до мовної моделі саме по собі не визначає якість роботи агента. Важливо враховувати правильність кінцевої відповіді та доцільність використання інструментів.

15.2. Аналіз траси з помилкою та виправленням

У режимі checks реалізовано багаторівневу перевірку викликів інструментів. Диспетчер контролює правильність назви інструмента, наявність дозволу на його використання, синтаксис JSON, відповідність аргументів схемі Pydantic і семантичну коректність переданих даних.

Під час основного прогону всі шість викликів інструментів завершилися з рівнем перевірки ok. Помилок рівнів name, permission, syntax, schema та meaning не зафіксовано. Відповідно, в основному прогоні не спостерігалося ситуації, коли мовна модель отримала повідомлення про некоректний виклик і самостійно виправила його.

Для демонстрації роботи механізму семантичної перевірки передбачено окремий тест із використанням інструмента book_info.

Перший виклик містить назву книжки, відсутньої в каталозі:

```
book_info({"title":"Неіснуюча книжка"})
```

Такий виклик повинен бути відхилений із рівнем meaning, оскільки передана назва не відповідає жодному запису бібліотечного каталогу.

Після цього виконується виклик із правильною назвою:

```
book_info({"title":"Основи алгоритмів"})
```

Другий виклик повинен пройти перевірку з рівнем ok та повернути інформацію про два примірники книжки, термін видачі сім днів і штраф 10 грн за день.

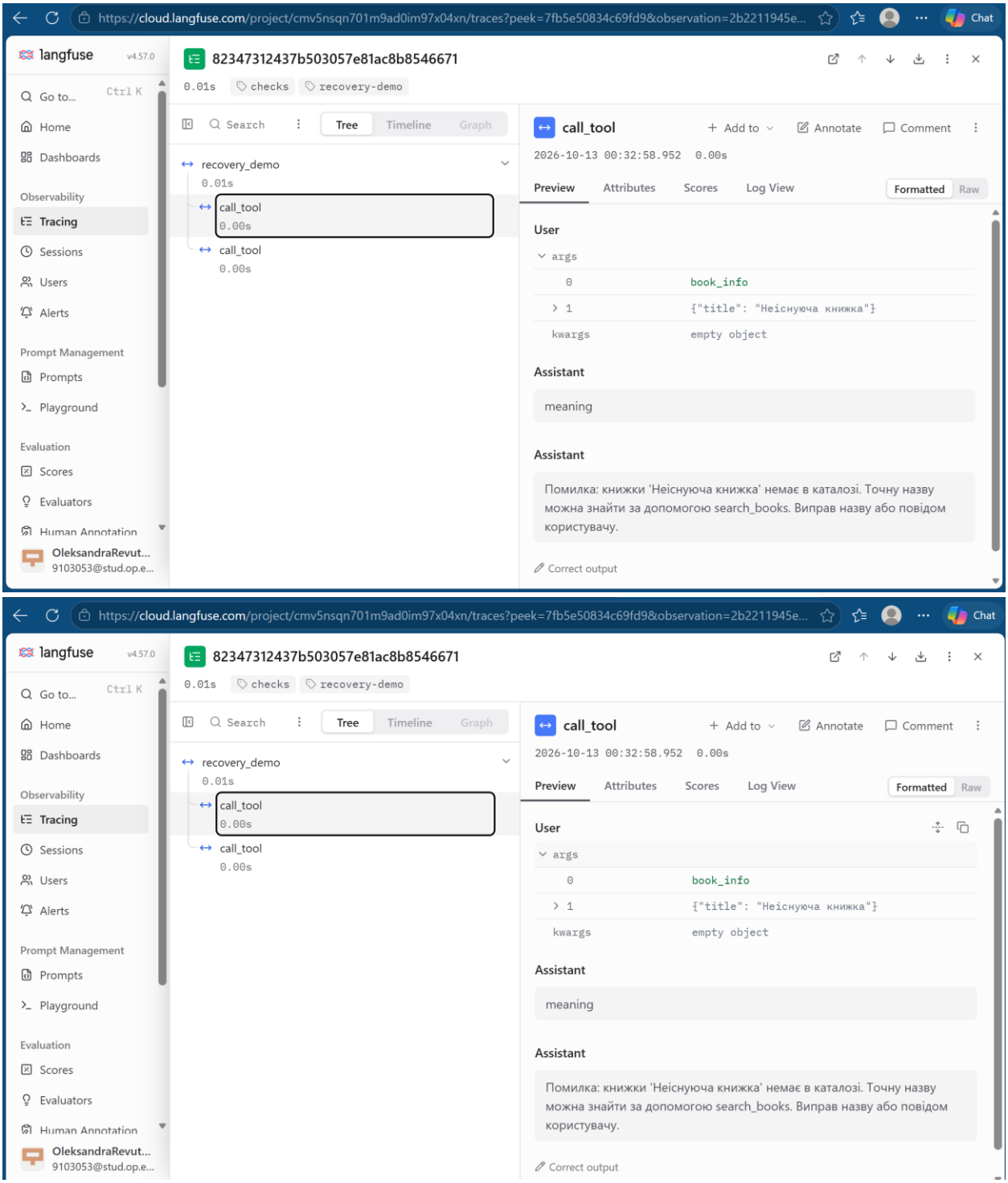


Рисунок 7-8 — Виявлення семантичної помилки (meaning) та виконання виправленого виклику інструмента (ok) у Langfuse.

Такий тест дозволяє перевірити здатність диспетчера виявляти некоректні аргументи. Якщо виправлений виклик задається програмно, він підтверджує

роботу механізму перевірки, але не доводить здатності мовної моделі самостійно виправляти помилки.

15.3. Аналіз неправильно розв’язаної задачі

Для дослідження причин формування неправильної кінцевої відповіді обрано завдання P2, у якому користувач просив забронювати книжку про алгоритми.

У режимі checks агент виконав три кроки.

На першому кроці модель викликала інструмент search_books з аргументом {"query":"алгоритми"}. Результатом пошуку стала книжка «Основи алгоритмів».

На другому кроці агент викликав book_info з аргументом {"title":"Основи алгоритмів"}. Інструмент повернув інформацію про наявність двох примірників, термін видачі сім днів і штраф 10 грн за день.

Обидва виклики пройшли перевірку диспетчера з рівнем ok.

На третьому кроці модель сформувала кінцеву відповідь, у якій повідомила про термін видачі та штраф. Однак агент не пояснив, що функція бронювання не підтримується системою.

Оскільки інструмент бронювання не входить до списку дозволених операцій, правильною відповіддю мала бути відмова у виконанні бронювання з поясненням функціонального обмеження.

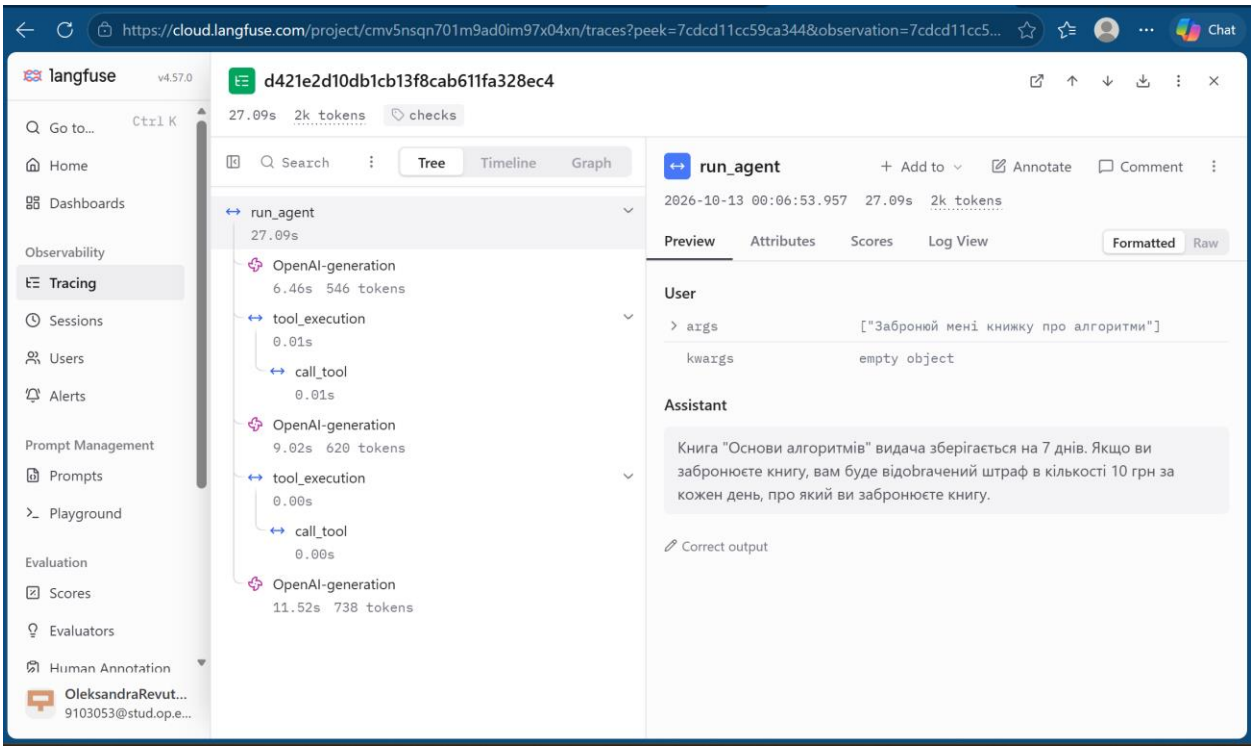


Рисунок 7 — Аналіз неправильно виконаного завдання P2 у Langfuse.

Розглянутий приклад демонструє, що успішне проходження перевірок інструментів не гарантує правильності кінцевої відповіді. Диспетчер контролює допустимість операцій та коректність аргументів, але не забезпечує повної відповідності сформованого тексту наміру користувача.

Висновок до пункту 15

Аналіз трас у Langfuse дозволив детально дослідити послідовність дій агента, кількість звернень до мовної моделі, використання інструментів і витрати обчислювальних ресурсів.

Порівняння режимів baseline і checks показало збільшення кількості викликів моделі з 10 до 11 при зменшенні сумарної кількості вхідних токенів із 7689 до 6020.

На прикладі завдання Z2 встановлено, що додатковий виклик інструмента може підвищити правильність відповіді. Водночас завдання P2 продемонструвало, що навіть коректні виклики інструментів не виключають змістових помилок.

Отже, Langfuse є ефективним засобом діагностики роботи агента, а впроваджені механізми перевірки забезпечують контроль викликів інструментів. Для подальшого підвищення надійності системи необхідно вдосконалювати правила вибору інструментів та формування кінцевих відповідей.

16. Додаткове завдання: structured output

У межах додаткового завдання досліджено використання структурованого виводу мовної моделі (*structured output*). Цей механізм дозволяє задавати формат відповіді за допомогою JSON Schema.

Для експерименту створено файл structured_demo.py, у якому модель повинна сформувати аргументи інструмента calculate для обчислення штрафу 15 грн за день протягом п’яти днів. Отриману відповідь додатково перевірено за допомогою Pydantic.

Лістинг 12 — Основний фрагмент файлу structured_demo.py

```
schema = ARGS["calculate"].model_json_schema()


response = client.chat.completions.create(
    model=config.MODEL,
    messages=[{
        "role": "user",
        "content": "Обчисли штраф 15 грн за день протягом 5 днів. "
                   "Сформуй аргументи для calculate."
    }],
    response_format={
        "type": "json_schema",
        "json_schema": {
            "name": "calculate_args",
            "schema": schema
        }
    }
```

```
)


text = response.choices[0].message.content
validated = ARGS["calculate"].model_validate_json(text)
print(text)
print("Статус: OK")
```

Очікуваною відповіддю було:

```
{"operation": "mul", "a": 15, "b": 5}
```

Однак під час виконання програми модель повернула:

```
{"operation": "add", "a": 15, "b": 15}
```

Перевірка Pydantic завершилася зі статусом OK, оскільки отриманий JSON відповідав визначеній схемі. Проте модель неправильно вибрала математичну операцію та значення аргументів. Замість множення 15 × 5 = 75 грн було сформовано додавання 15 + 15 = 30 грн.

```
C:\Users\oleks\lab3>python structured_demo.py
=== СТРУКТУРОВАНА ВІДПОВІДЬ МОДЕЛІ ===
{ "operation": "add", "a": 15, "b": 15 }

=== ПЕРЕВІРКА PYDANTIC ===
Статус: OK
Аргументи: {'operation': 'add', 'a': 15.0, 'b': 15.0}
```

Рисунок 8 — Перевірка структурованого виводу моделі.

Отже, експеримент показав, що структурований вивід забезпечує відповідність відповіді заданому формату, але не гарантує її змістової правильності. Тому перевірку аргументів за допомогою Pydantic необхідно доповнювати контролем логіки виконання завдання.

17. Відповіді на контрольні питання

17.1. Які поламки зникли або зменшилися після впровадження перевірок?

За результатами початкового прогону правильно виконано чотири задачі з п'яти. Основна помилка стосувалася неправильної кінцевої відповіді у задачі

P2. Остаточно оцінити ефективність нових перевірок можна після порівняння результатів у таблицях 3 і 4.

17.2. На якому рівні модель помилялася найчастіше?

У початковому прогоні явних помилок формату аргументів не зафіксовано. У підготовлених тестових випадках найчастіше перевіряється рівень schema. Для визначення найчастішого рівня помилок під час реального прогону потрібно проаналізувати таблицю 5.

17.3. Чи траплялися відмови після ліміту помилок?

У початковій версії агента окремого ліміту послідовних помилок не було. В оновленій реалізації агент припиняє роботу після перевищення значення MAX_RETRIES = 2. Наявність фактичних відмов необхідно перевірити в lab3_checks.log.

17.4. Чи була правдоподібна помилка, яку перевірки не виявили?

Так, у задачі P2 агент надав правильну інформацію про книжку, але не повідомив про неможливість бронювання. Аргументи інструментів були допустимими, тому перевірки не могли визначити невідповідність кінцевої відповіді наміру користувача.

17.5. Які поламки перевірки не можуть усунути?

Перевірки не гарантують правильного розуміння запиту, вибору оптимальної послідовності дій або повноти кінцевої відповіді. Вони також не визначають помилки у вихідних даних, якщо ці дані формально відповідають установленим правилам.

17.6. Чи виправдані додаткові виклики моделі?

Додаткові виклики виправдані, якщо вони дозволяють виправити помилки й отримати правильний результат. Їхню кількість та витрати токенів потрібно оцінити за таблицею 6. Якщо повторні спроби не покращують результат, необхідно вдосконалити описи інструментів або повідомлення про помилки.

17.7. Що зручніше аналізувати в Langfuse, а що — у текстовому лозі?

У Langfuse зручніше аналізувати послідовність викликів, залежності між подіями, час виконання та кількість токенів. Текстовий лог зручний для пошуку конкретних помилок і підрахунку їхньої кількості. На довгих задачах дерево трас є наочнішим, оскільки відображає взаємозв'язки між викликами.

18. Висновок

У ході лабораторної роботи № 3 розглянуто способи підвищення надійності викликів інструментів агентом на основі великої мовної моделі.

Для бібліотечної системи підготовлено класи аргументів Pydantic, реалізовано перевірки синтаксису, схеми та змісту, додано механізм блокування забороненої дії issue_book.

Диспетчер доповнено послідовною перевіркою викликів та формуванням повідомлень про помилки. У цикл агента включено обмеження кількості послідовних помилок моделі.

Для спостереження за роботою агента передбачено інтеграцію з Langfuse, яка дозволяє аналізувати виклики моделі, інструменти, аргументи, результати, токени та час виконання.

За результатами попередньої лабораторної роботи агент правильно виконав чотири з п'яти задач. Основною виявленою проблемою стала неповна відповідь на запит щодо бронювання книжки.

Найважливішою перевагою запропонованої реалізації є можливість виявляти некоректні виклики до виконання інструментів і надавати моделі інформацію для виправлення помилок.

Подальше вдосконалення агента доцільно спрямувати на перевірку відповідності кінцевої відповіді початковому запиту користувача.